

PRODUCT SALES REGIONAL ANALYSIS

Project Documentation — Source Edition

Built from verified Excel file | Nigerian Retail Sales Scenario | 2024 Dataset

Analyst: Akinwale David | 2025

Tool: Microsoft Excel | File: Product-Sales-Region_Data.xlsx

1,500	23	5	4	3
Orders	Columns	Regions	Biz Questions	Promotions

Table of Contents

1. Dataset Overview.....	4
2. Data Cleaning Process	5
2.1 Checking for Duplicates.....	5
2.2 Standardising Columns	6
2.3 Data Cleaning Summary.....	7
Question 1: Revenue by Region & Customer Type	8
3.1 Objective & Approach.....	8
3.2 PivotTable Configuration	8
3.3 DAX Measures & Why	8
Total Revenue	8
Retail Revenue & Wholesale Revenue	8
% of Grand Total.....	9
3.4 Visualisation Choice: Clustered Bar Chart	9
3.5 Findings.....	9
Question 2: Salesperson Sales Volume & Return Rate.....	11
4.1 Objective & Approach.....	11
4.2 PivotTable Configuration	11
4.3 DAX Measures & Why	11
Return Rate	11
Revenue Rank & Return Rate Rank	12
4.4 Visualisation Choice: Combo Chart (Column + Line)	12
4.5 Findings.....	12
Question 3: Promotion Effectiveness Analysis	13
5.1 Objective & Approach.....	13
5.2 PivotTable Configuration	13
5.3 DAX Measures & Why	13
Average Order Value per Promotion	13
Revenue Lift vs. No-Promotion Baseline.....	14
5.4 Visualisation Choice: Clustered Column Chart	14
5.5 Findings.....	14
Question 4: Delivery Time & Delay Rate by Region	16
6.1 Objective & Approach.....	16
6.2 PivotTable Configuration	16
6.3 DAX Measures & Why	16
Delayed Orders Count	16
Delay Rate.....	17

6.4 Visualisation Choice: Combo Chart (Bar + Line)..... 17

6.5 Findings..... 17

1. Dataset Overview

This project analyses a regional product sales dataset for a Nigerian retail business, built entirely in Microsoft Excel using PivotTables and DAX measures. The source file is Product-Sales-Region_Data.xlsx, containing 1,500 order records across 23 fields spanning five geographic regions, six salespersons, seven product categories, and three promotional campaigns.

Attribute	Detail
File Name	Product-Sales-Region_Data.xlsx
Total Records	1,500 orders
Total Columns	23 fields
Regions	East, West, North, South, Central
Salespersons	Alice, Bob, Carlos, Diana, Eva, Frank
Products	Laptop, Phone, Desk, Chair, Monitor, Tablet, Printer
Promotions	FREESHIP, SAVE10, WINTER15, None
Customer Types	Retail, Wholesale
Payment Methods	Online, Credit Card, Debit Card, Gift Card, Cash
Key Columns	TotalPrice, Quantity, Discount, Returned, OrderDate, DeliveryDate, Delivery Days, Delivery Status
Tool	Microsoft Excel — PivotTables, DAX Measures (Data Model)
Region Managers	Eric (East), Sophie (South), Ryan (North), Cameron (Central), Wendy (West)

2. Data Cleaning Process

Before any PivotTables or DAX measures were built, the dataset in Product-Sales-Region_Data.xlsx was inspected and cleaned inside Excel. The cleaning process focused on two core activities: checking for duplicate records and standardising column values to ensure consistent grouping and aggregation throughout the analysis.

2.1 Checking for Duplicates

Duplicate records can silently distort every metric in an analysis — inflating revenue totals, overstating order counts, and producing misleading fraud or return rates. The following checks were applied to the dataset before any analysis began:

Step 1 — Check for Duplicate OrderIDs
In Excel: Select the OrderID column → Home → Conditional Formatting → Highlight Cell Rules → Duplicate Values This visually flags any OrderID that appears more than once in the dataset. Alternatively, a helper column was added: =COUNTIF(\$A\$2:\$A\$1501, A2) — any value greater than 1 flags a duplicate OrderID. → Why check OrderID first? OrderID is the primary unique identifier for each transaction. A duplicated OrderID means the same order is being counted twice, which would inflate Total Revenue, order counts, and return rates across every PivotTable. → Result: No duplicate OrderIDs were found. All 1,500 rows represent distinct orders.
Step 2 — Check for Fully Duplicate Rows
In Excel: Data tab → Remove Duplicates → Select All Columns → OK Excel reports the number of duplicate rows found and removed, and the number of unique rows remaining. → Why check for full-row duplicates separately? A row can be fully duplicated even when the OrderID is not — for example if OrderID was left blank or if the same transaction was entered twice with the same details across all columns. → Result: Zero fully duplicate rows detected. The dataset confirmed clean at 1,500 unique records.
Step 3 — Verify Key Column Completeness (No Blanks)
In Excel: Use Ctrl + End to confirm the data range, then filter each key column for blanks: Data → Filter → Click dropdown on column → Uncheck 'Select All' → Check 'Blanks' Key columns checked: OrderID, Region, Salesperson, TotalPrice, CustomerType, Returned, OrderDate, DeliveryDate

→ Why check for blanks? A blank in Region or Salesperson would cause PivotTable rows to show as '(blank)' — an unintended grouping that distorts regional and salesperson summaries without raising an error.

→ Result: No blanks found in any key analytical column. All 1,500 rows are complete.

2.2 Standardising Columns

Column standardisation ensures that categorical fields use consistent formatting so that PivotTable GROUP BY operations and DAX filter contexts work correctly. Inconsistent casing or spacing causes Excel to treat the same value as two different categories — for example, 'retail' and 'Retail' would appear as two separate rows in a PivotTable.

Step 4 — Standardise Text Casing Across Categorical Columns

Columns checked: Region, CustomerType, Salesperson, Promotion, Delivery Status, PaymentMethod

In Excel: Use Find & Replace (Ctrl+H) to correct any inconsistent casing observed during visual inspection.

Alternatively, a helper column with =PROPER(A2) or =UPPER(A2) can be used to preview the standardised value before replacing.

→ Why PROPER case for names (Salesperson, Region)? PROPER capitalises the first letter of each word — consistent with how names and place names should appear in reports and charts.

→ Why check Promotion column specifically? The Promotion column contains both text codes (FREESHIP, SAVE10, WINTER15) and blank cells for orders with no promotion. Blank cells were confirmed to be genuinely empty rather than containing spaces, ensuring AVERAGEIF and PivotTable grouping treat them correctly as 'No Promotion'.

→ Result: All categorical columns confirmed to use consistent formatting throughout.

Step 5 — Validate the Returned Column Encoding

The Returned column uses binary encoding: 1 = order was returned, 0 = order was not returned.

In Excel: Data → Filter → Returned column → check that only the values 0 and 1 appear. No text, no blanks, no other numbers.

→ Why validate this encoding? The Return Rate DAX measure uses SUM(Returned) and COUNTROWS to calculate the proportion of returned orders. If the column contains any text values ('Yes'/'No') or blanks, SUM() would silently return 0 for those rows, understating the true return count.

→ Result: Returned column confirmed to contain only binary values (0 and 1) across all 1,500 rows.

Step 6 — Confirm Date Column Formats

Columns: OrderDate and DeliveryDate

In Excel: Select each date column → Home → Number format dropdown → confirm 'Date' format, not 'General' or 'Text'.

A date stored as text will not sort correctly in PivotTables and cannot be used in date arithmetic.

The Delivery Days column (DeliveryDate – OrderDate) was also verified to be formatted as 'Number' (integer days), not as a Date serial.

→ Why verify Delivery Days format? If Delivery Days is formatted as a Date, Excel displays it as a date like '04/01/1900' instead of the integer 4. This would make the AVERAGE aggregation in PivotTables return a meaningless date rather than an average number of days.

→ Result: OrderDate and DeliveryDate confirmed as Date format. Delivery Days confirmed as Number (integer). Delivery Status confirmed as text ('Timely' / 'Delayed') with consistent casing.

2.3 Data Cleaning Summary

Check Performed	Column(s)	Result	Action Taken
Duplicate OrderIDs	OrderID	None found	No action required
Fully duplicate rows	All columns	None found	No action required — 1,500 unique records confirmed
Blank values in key columns	OrderID, Region, TotalPrice etc.	Zero blanks	No action required — all key columns complete
Text casing consistency	Region, CustomerType, Promotion	Consistent formatting	Confirmed via visual inspection and Find & Replace
Returned column encoding	Returned	Binary 0/1 only	No action required — encoding confirmed clean
Date column format	OrderDate, DeliveryDate	Date format confirmed	No action required
Delivery Days format	Delivery Days	Number (integer) confirmed	No action required — no date serial issues

Question 1: Revenue by Region & Customer Type

Business Question: Which region generates the highest total revenue, and how does customer type (Retail vs. Wholesale) influence that breakdown?

3.1 Objective & Approach

The goal was to aggregate order-level revenue (TotalPrice) across two dimensions simultaneously: Region and CustomerType. A PivotTable was built on the source data in Product-Sales-Region_Data.xlsx, placing Region in rows, CustomerType in columns, and TotalPrice in the Values area as a Sum. DAX measures were then written in the Data Model to enable dynamic filtering and percentage calculations that PivotTables alone cannot produce natively.

3.2 PivotTable Configuration

PivotTable Setup
Source: Product-Sales-Region_Data.xlsx — Table1 (all 1,500 rows)
Rows: Region field (East, West, North, South, Central)
Columns: CustomerType field (Retail, Wholesale)
Values: TotalPrice — summarised as Sum, formatted as Currency
→ Why Sum and not Count or Average? The business question asks which region generates the most revenue — Sum of TotalPrice is the direct answer. Count would measure order frequency, Average would measure order size; neither answers the revenue question.
→ Why CustomerType in Columns? Placing it in columns automatically creates side-by-side subtotals for Retail and Wholesale within every region row, eliminating the need to build separate tables for each segment.

3.3 DAX Measures & Why

Total Revenue

DAX Measure
Total Revenue = SUM(Table1[TotalPrice])
→ The base aggregation measure. SUM is used rather than reading the PivotTable's implicit sum because a named DAX measure can be referenced by other measures (e.g., % of Total) and persists across filter contexts.
→ TotalPrice is chosen over UnitPrice × Quantity because TotalPrice already incorporates the Discount applied — it is the actual revenue received, not the list price.

Retail Revenue & Wholesale Revenue

DAX Measure

Retail Revenue = CALCULATE([Total Revenue], Table1[CustomerType] = "Retail")

Wholesale Revenue = CALCULATE([Total Revenue], Table1[CustomerType] = "Wholesale")

→ CALCULATE wraps the base measure and overrides the filter context with a specific CustomerType value. This forces the measure to return only Retail or Wholesale revenue regardless of what slicers or PivotTable filters are active.

→ Why CALCULATE instead of a PivotTable column filter? CALCULATE measures can be placed in separate columns of the same PivotTable, enabling direct side-by-side comparison of Retail vs. Wholesale for every region in a single view.

% of Grand Total**DAX Measure**

% of Grand Total = DIVIDE([Total Revenue], CALCULATE([Total Revenue], ALL(Table1[Region])), 0)

→ DIVIDE is used instead of a direct division operator to handle zero denominators gracefully — returning 0 rather than an error if a region has no sales.

→ ALL(Table1[Region]) removes the region filter context from the denominator, forcing it to sum all regions regardless of which row is currently in scope. Without ALL(), the denominator would equal the numerator for every row, returning 100% everywhere.

→ This metric answers a different but equally important sub-question: not just 'how much?' but 'how significant is this region relative to the whole?'

3.4 Visualisation Choice: Clustered Bar Chart

A clustered bar chart was built from the PivotTable using Insert → PivotChart → Bar → Clustered Bar. This chart type was chosen because:

- It shows absolute revenue values — not just proportions — so the magnitude difference between regions is immediately clear.
- The clustered format places Retail and Wholesale bars side-by-side for every region, making the segment split visible without toggling between views.
- Horizontal bar orientation allows region labels to be read cleanly on the Y-axis without text rotation.

3.5 Findings

Region	Retail Revenue	Wholesale Revenue	Total Revenue	% of Total
North	\$522,368	\$445,590	\$967,958	22.1%
East	\$417,764	\$465,869	\$883,634	20.2%
West	\$399,793	\$453,686	\$853,479	19.5%
Central	\$474,462	\$372,691	\$847,154	19.3%
South	\$381,137	\$446,631	\$827,768	18.9%

Region	Retail Revenue	Wholesale Revenue	Total Revenue	% of Total
TOTAL	\$2,195,525	\$2,184,467	\$4,379,992	100%

 **Key Findings**

- North leads at \$967,958 — approximately 17% ahead of South (\$827,768), the lowest-performing region.
- Retail dominates in North, Central, and East; Wholesale edges ahead in South and West.
- Central has the widest Retail–Wholesale gap (\$101K), suggesting stronger brick-and-mortar or direct consumer presence.
- Revenue is well balanced across all five regions — no single region exceeds 22.1% of total, indicating healthy geographic spread.

Question 2: Salesperson Sales Volume & Return Rate

Business Question: Which salesperson has the highest sales volume and the lowest return rate?

4.1 Objective & Approach

This question requires two separate but related analyses: how much each salesperson sells (volume) and how often their orders are returned (quality). Answering with revenue alone would ignore quality; answering with return rate alone would ignore contribution. A PivotTable was used to aggregate both metrics per salesperson, with DAX measures computing the normalised return rate and ranking both dimensions.

4.2 PivotTable Configuration

PivotTable Setup

Source: Product-Sales-Region_Data.xlsx — Table1

Rows: Salesperson field

Values: TotalPrice (Sum → Total Revenue), OrderID (Count → Total Orders), Returned (Sum → Total Returned)

→ Why Count of OrderID for orders? OrderID is a unique identifier per transaction, so counting it gives the exact number of orders per rep — more reliable than counting TotalPrice rows which could theoretically have duplicates.

→ Why Sum of Returned? The Returned column is binary encoded (1 = returned, 0 = not returned), so summing it directly produces the count of returned orders. No additional filter table is needed.

4.3 DAX Measures & Why

Return Rate

DAX Measure

Return Rate = DIVIDE(SUM(Table1[Returned]), COUNTROWS(Table1), 0)

→ Return Rate normalises across salespersons who handle different order volumes. Bob has 64 returns from 243 orders (26.3%) while Carlos has 76 returns from 270 orders (28.1%) — the raw counts suggest Carlos is closer to Bob, but the rates reveal Carlos has a materially worse quality problem.

→ DIVIDE is used instead of direct division to handle any salesperson with zero orders without returning an error.

→ COUNTROWS(Table1) counts orders in the current filter context (i.e., for the current salesperson row), serving as the denominator.

→ Why rate over count? Without normalisation, high-volume salespersons would always appear to have more returns in absolute terms, masking quality issues in lower-volume reps.

Revenue Rank & Return Rate Rank

DAX Measures
Revenue Rank = RANKX(ALL(Table1[Salesperson]), [Total Revenue],, DESC, DENSE)
Return Rate Rank = RANKX(ALL(Table1[Salesperson]), [Return Rate],, ASC, DENSE)
→ RANKX iterates over all salesperson values (ALL() removes the current row filter) and ranks the current salesperson's measure value against all others.
→ DESC for Revenue Rank: rank 1 = highest revenue — the desired top performer.
→ ASC for Return Rate Rank: rank 1 = lowest return rate — the desired quality leader. Opposite sort direction to Revenue Rank is deliberate.
→ DENSE ranking ensures no gaps in rank numbers when values are tied.
→ Ranking converts two metrics on completely different scales into a common 1-to-6 ordinal scale, making dual comparison instant.

4.4 Visualisation Choice: Combo Chart (Column + Line)

Revenue and return rate cannot share the same Y-axis — they operate on incompatible scales. A combo chart was built from the PivotChart menu: revenue as clustered columns on the primary axis, return rate as a line on a secondary axis. This keeps both metrics readable in a single view without distorting either.

4.5 Findings

Salesperson	Total Revenue	Orders	Returned	Return Rate	Rev Rank	RR Rank
Bob	\$796,781	243	64	26.34%	1	5
Alice	\$786,166	259	64	24.71%	2	2
Frank	\$714,642	243	55	22.63%	3	2
Carlos	\$707,167	270	76	28.15%	4	6
Eva	\$698,670	236	51	21.61%	5	1
Diana	\$676,568	249	62	24.90%	6	3

 Key Findings
• Bob leads in revenue (\$796,781) but ranks 5th in quality at a 26.34% return rate — high volume with a quality cost.
• Eva has the lowest return rate (21.61%) and is the most reliable closer despite ranking 5th in revenue.
• Carlos has the highest return rate at 28.15% — nearly 1 in 3.5 orders is returned, a significant risk to net revenue.
• Alice is the most balanced performer: 2nd in revenue and 2nd in return rate quality — the strongest all-round rep.

Question 3: Promotion Effectiveness Analysis

Business Question: Do promotions actually increase order value, or do discounts just erode revenue?

5.1 Objective & Approach

Promotions carry a built-in tension: they are designed to drive more orders or higher spending, but the discount cost reduces the revenue collected per order. The analysis quantified both sides — what each promotion costs (average discount rate) and what it returns (average order value, quantity, total revenue) — and compared each promotion type against the no-promotion baseline using a PivotTable and DAX.

5.2 PivotTable Configuration

PivotTable Setup
Source: Product-Sales-Region_Data.xlsx — Table1
Rows: Promotion field (FREESHIP, SAVE10, WINTER15, blank → grouped as No Promotion)
Values: TotalPrice (Sum → Total Revenue), TotalPrice (Average → Avg Order Value), Discount (Average → Avg Discount Rate), Quantity (Average → Avg Quantity)
→ Why both Sum and Average of TotalPrice? Sum answers 'which promotion generated the most total revenue?' while Average answers 'did this promotion make individual customers spend more per transaction?' — the latter is the core effectiveness question.
→ Why Average of Quantity? A promotion might not raise the price per order but could succeed by driving customers to buy more units. Avg Quantity reveals whether the mechanism is volume-based or value-based.
→ Blank Promotion values were grouped in the PivotTable as 'No Promotion' to create a clean baseline row.

5.3 DAX Measures & Why

Average Order Value per Promotion

DAX Measure
Avg Order Value = AVERAGE(Table1[TotalPrice])
→ This is the single most important metric for the question. Total revenue could be high simply because a promotion appeared on more orders — not because it drove higher spending per transaction.
→ Average Order Value (AOV) isolates the per-transaction effect, answering whether customers who used FREESHIP spent more per order than those with no promotion code.

→ Evaluated within the PivotTable's Promotion row filter context, this measure automatically returns the AOV for each promotion group.

Revenue Lift vs. No-Promotion Baseline

DAX Measure

No Promo AOV = CALCULATE([Avg Order Value], Table1[Promotion] = "")
Revenue Lift = [Avg Order Value] - [No Promo AOV]

→ Revenue Lift is the most direct answer to the business question: is this promotion worth running?
→ CALCULATE with Promotion = blank isolates the no-promotion baseline regardless of the current row filter, making it a fixed reference point for all comparisons.
→ A positive lift means the promotion pushes customers to spend above what they would without a code. A negative lift means the discount costs more than it recovers in incremental volume.

5.4 Visualisation Choice: Clustered Column Chart

A clustered column chart was built from the PivotChart, placing Avg Order Value and Avg Discount Rate side-by-side for each promotion group. This makes the cost-benefit trade-off immediately visible: a tall revenue bar alongside a short discount bar signals an efficient promotion; equal heights signal value erosion.

5.5 Findings

Promotion	Orders	Avg Order Value	Total Revenue	Avg Discount	Avg Qty	Revenue Lift
FREESHIP	419	\$2,954.84	\$1,238,077	7%	10.5	+\$51.47
SAVE10	338	\$2,927.98	\$989,657	8%	10.7	+\$24.61
No Promotion	370	\$2,903.37	\$1,074,247	7%	10.2	—
WINTER15	373	\$2,890.11	\$1,078,012	7%	10.3	-\$13.26

 Key Findings

- FREESHIP is the best-performing promotion: highest Avg Order Value (\$2,954.84), highest total revenue (\$1.24M), and highest average quantity per order (10.5 units).
- FREESHIP and SAVE10 both produce positive revenue lift vs. the no-promotion baseline, meaning they drive incremental spending.
- WINTER15 produces a negative lift (-\$13.26) — customers with WINTER15 actually spend slightly less per order on average than those with no promotion, making it the weakest campaign.
- SAVE10 carries the highest average discount rate (8%) but only the second-lowest AOV lift — the least efficient promotion per percentage point discounted.

- FREESHIP outperforms because it removes a psychological barrier (shipping cost) without cutting the product price, preserving perceived value while increasing conversion.

Question 4: Delivery Time & Delay Rate by Region

Business Question: What is the average delivery time by region, and which region has the most delayed orders?

6.1 Objective & Approach

Unlike the previous questions, the dataset already contains two pre-computed delivery columns: Delivery Days (integer count of days between OrderDate and DeliveryDate) and Delivery Status (categorical: 'Timely' or 'Delayed'). The analysis used these columns directly in a PivotTable, with DAX measures calculating the delay rate as a normalised comparison across regions of different sizes.

6.2 PivotTable Configuration

PivotTable Setup
Source: Product-Sales-Region_Data.xlsx — Table1
Rows: Region field
Values: Delivery Days (Average → Avg Delivery Days), Delivery Days (Max → Max Delivery Days), OrderID (Count → Total Orders), Returned (Sum → Returned Orders)
Filter / Slicer: Delivery Status = 'Delayed' on a second PivotTable to count delayed orders per region
→ Why use the existing Delivery Days column rather than recalculating? The dataset pre-computes Delivery Days, already formatted as an integer. Using it directly eliminates the risk of date subtraction formatting errors.
→ Why Average of Delivery Days and not Median? The Delivery Days values range from 1 to 10 — a bounded, near-uniform distribution where average and median are very close. Average is natively supported by PivotTable aggregation without DAX.
→ Why Max of Delivery Days? Identifies the single worst delivery in each region and reveals whether outliers skew the average. When all regions share the same maximum (10 days), it signals a system ceiling rather than a region-specific failure.

6.3 DAX Measures & Why

Delayed Orders Count

DAX Measure
Delayed Orders = CALCULATE(COUNTROWS(Table1), Table1[Delivery Status] = "Delayed")
→ CALCULATE restricts COUNTROWS to only rows where Delivery Status = 'Delayed', evaluated within the current Region filter context from the PivotTable row.
→ The Delivery Status column is used as the delay flag rather than comparing Delivery Days to a threshold, because it is the dataset's own classification — ensuring consistency with how delays were defined at source.

→ COUNTROWS is used instead of SUM because Delivery Status is a text column, not numeric; COUNTROWS counts qualifying rows regardless of their value.

Delay Rate

DAX Measure

Delay Rate = DIVIDE([Delayed Orders], COUNTROWS(Table1), 0)

→ Delay Rate normalises delayed order counts across regions with different total order volumes. North (309 orders, 163 delayed = 52.75%) is correctly identified as worse than West (284 orders, 122 delayed = 42.96%), even though West has fewer delayed orders in absolute terms.

→ Without normalisation, a region simply handling more orders would appear to have a worse delay problem due to scale — not due to logistics efficiency.

→ DIVIDE handles the zero-denominator case safely.

→ The result is formatted as a percentage in the PivotTable via Value Field Settings → Number Format → Percentage.

6.4 Visualisation Choice: Combo Chart (Bar + Line)

Average delivery days and delay rate cannot share the same Y-axis due to different scales. A PivotChart combo was configured with Avg Delivery Days as horizontal bars on the primary axis and Delay Rate as a line on a secondary axis — both metrics remain clearly readable in a single, uncluttered view.

6.5 Findings

Region	Avg Delivery Days	Max Days	Total Orders	Delayed Orders	Delay Rate	Returns
North	6.39	10	309	163	52.75%	70
West	6.01	10	284	122	42.96%	74
East	6.00	10	311	138	44.37%	81
South	5.93	10	295	130	44.07%	77
Central	5.88	10	301	128	42.52%	70

 Key Findings

- North is the worst-performing delivery region: highest average delivery days (6.39) and the highest delay rate (52.75%) — more than half of all North orders arrive later than the dataset average.
- Central is the most efficient region: lowest average delivery days (5.88) and lowest delay rate (42.52%).
- All five regions share the same maximum delivery of 10 days — the ceiling is a system-wide constraint, not a region-specific outlier problem.
- The range between fastest and slowest region is only 0.51 days on average, but translates to a meaningful 10-percentage-point gap in delay rates between North and Central.

- East has the most returned orders (81) despite being roughly average on delivery speed, suggesting returns in that region may be driven by factors beyond delivery time.

Summary: Methods & DAX Measures Used

Question	Method	Primary DAX Measure	Reason for Choice
Q1 – Revenue by Region	PivotTable + DAX	CALCULATE with CustomerType filter	Isolates Retail vs. Wholesale in a single PivotTable column without separate tables
Q1 – % of Total	DAX	DIVIDE + ALL() on Region	ALL() removes row filter so denominator covers all regions, not just current row
Q2 – Revenue	PivotTable + DAX	SUM(TotalPrice)	Post-discount actual revenue, not list price × quantity
Q2 – Return Rate	DAX	DIVIDE(SUM(Returned), COUNTROWS, 0)	Normalises across reps with different order volumes; DIVIDE prevents zero errors
Q2 – Ranking	DAX	RANKX with ALL() + ASC/DESC sort	Converts two incompatible scales to a common 1-to-6 ordinal for dual comparison
Q3 – Avg Order Value	PivotTable + DAX	AVERAGE(TotalPrice)	Per-transaction metric isolates the promotion's spending effect from volume effects
Q3 – Revenue Lift	DAX	CALCULATE with blank Promotion filter	Fixed baseline reference regardless of current PivotTable row context
Q4 – Avg Delivery Days	PivotTable	AVERAGE(Delivery Days) — native	Pre-computed column used directly; bounded distribution makes average appropriate
Q4 – Delay Rate	DAX	DIVIDE(Delayed Orders, COUNTROWS, 0)	Normalises delayed counts across regions of different sizes
Q4 – Delayed Orders	DAX	CALCULATE(COUNTROWS, Status=Delayed)	Uses dataset's own Delivery Status classification for consistency

Every method and DAX measure in this analysis was chosen to answer a specific business question accurately and dynamically. The guiding principle throughout: use the aggregation that reflects the actual business metric (post-discount revenue, normalised rates), apply DAX's filter context control (CALCULATE, ALL, DIVIDE) to produce measures that remain correct under any PivotTable slice or drill-down, and cross-reference related metrics (delay rate vs. returns, revenue rank vs. return rate rank) to surface the business insight rather than just the number.